
D3481 CLUB MANAGEMENT SYSTEM

D3481 扶輪社務管理平台建置歷程

從 0 到現在：一套系統怎麼跟 AI 一輪一輪對出來

2026-07-07 整理

目錄

1. 起點：3481 地區的 AI 元年

2. 為什麼不外包，選擇跟 AI 一輪一輪對出來

3. 第一份文件：怎麼寫規格書跟 AI 溝通

4. 技術架構定案與理由

4.1 角色體系

4.2 貫穿全系統的一條設計原則：各社管各社，地區只管地區

4.3 Feature Flag：讓地區決定要不要開放某個功能

4.4 資料庫表格清單（Phase 1 完整版）

5. 開發歷程：七個階段的軌跡

階段一 | 地基：帳號、權限、核心四大功能（早期輪次）

階段二 | 正式站部署除錯（Cloudflare Pages）

階段三 | 帳號邀請機制打磨

階段四 | 降低使用門檻：自助註冊、手機號碼帳號、地區唯讀角色

階段五 | 視覺改版：從左側選單到頂部橫向選單

階段六 | 社史資料與跨社協作

階段七 | 社友活動報名系統

6. 真實踩過的坑：五個代表性案例

案例一：登入後系統「失憶」——RLS 無窮遞迴

案例二：handle_new_user() 讀錯 metadata 欄位——影響範圍最大的一個 bug

案例三：帳號邀請權限模型設計成單向階層，卡出死結

案例四：Edge Function 的網址 slug 跟你以為的名字對不上

案例五：Cloudflare Pages 一開始從來沒有真正 build 成功過

7. 一路走來的方法論

8. 現況總覽

9. 下一步 (Phase 2)

10. 給下一個想做類似系統的人

1. 起點：3481 地區的 AI 元年

這整個計劃，是 3481 地區 2026–27 年度總監 Computer 提出的。他把這一年定調為「**3481 地區 AI 元年**」。

他的想法是：AI Agent 這幾年進步得很快，已經到了可以真正協助扶輪社處理各種工作的程度——不只是社務管理數位化、知識庫建立、活動記錄與統計，還包括對外的公共關係拓展：影片製作、活動紀錄文案、獎助金申請等等。目的不是「用 AI 取代人」，而是把原本要靠人工一步一步做的流程，變成更輕鬆便利的事情，讓有限的人力資源可以放在真正需要人去判斷、去經營關係的地方。

這一年度的主題是「**Create Lasting Impact，創造持恆的影響力**」。這套社務管理系統，是這個願景底下第一個真正落地、跑在正式站上的具體成果。

國際扶輪 3481 地區底下有超過 100 個社。每個社自己管自己的社友名冊、例會出席、幹部異動、公告——大部分靠 Excel、LINE 群組、紙本簽到，加上執秘的個人記憶。社友不多的時候還撐得住，但地區要彙整全區數據（誰的出席率掉了、哪個社的幹部資料過期了、總監獎項申請進度到哪）的時候，就變成每次都要跟 100 多個社——追進度、對格式、抓錯字。

2. 為什麼不外包，選擇跟 AI 一輪一輪對出來

一般的解法是找廠商外包一套系統：花錢、花時間、還要來回開會講需求，需求講不清楚就等下一版。這次選的是另一條路：直接跟 AI 對話，把系統一輪一輪對出來。

很多人聽到「AI 做系統」，想到的畫面是打一句話、AI 生出一個成品。實際上完全不是這樣——這比較接近「找一個工程師夥伴」，只是溝通管道是文字對話，而且回應速度是幾分鐘而不是幾週。

整個開發到目前為止累積了**四十多輪對話**，每一輪大致是同樣的循環：

1. 提出一個需求或回報一個問題（例如「執秘登入後選單都是空的」「有個帳號同時要管兩個社」）
2. AI 去看現有的程式碼，找出真正的原因
3. AI 提出修法，講清楚取捨（改哪裡、為什麼、有沒有副作用）
4. 一起確認、驗證、把結果記錄下來

這個循環被完整記錄在 `HANDOFF.md` 裡——與其說是一份技術文件，它更像是一本開發日記，方便下一輪接續，也方便任何人回頭理解「這個系統為什麼會長成現在這樣」。

3. 第一份文件：怎麼寫規格書跟 AI 溝通

在第一行程式碼寫出來之前，先花時間寫了一份完整的規格書：`CODEX_PROMPT.md`。這份文件本身就是「想清楚這個系統到底要解決什麼問題」之後的產物，也是整個專案跟 AI 溝通的起點。

一份能讓 AI 有效執行的規格書，至少要講清楚四件事：

(1) 專案概述——一句話講清楚系統的邊界

「為國際扶輪3481地區建置一套社務管理平台。支援100+個扶輪社，每社有多位使用者，地區管理員可跨社查看資料。」

(2) 技術棧——明確指定，不留給 AI 自由發揮

項目	選擇
框架	Vue 3 + Vite + TypeScript (Composition API)
路由	Vue Router 4
狀態管理	Pinia
後端	Supabase (Auth + PostgreSQL + RLS)
Excel	SheetJS (xlsx)
部署	Cloudflare Pages
CSS	不用 Tailwind，用既有 CSS variables 設計系統

(3) 功能範圍——用代號拆解成可獨立驗收的區塊

規格書把 Phase 1 拆成 A（帳號）、B（例會管理）、D（社友名冊）、H（通訊錄）四大區塊，每區塊底下再編號（B1 例會列表、B2 出席彙總、B3 個人出席率……）。這個編號習慣一路沿用到後來的 `ARCHITECTURE.md` 的 feature flag 命名（`B5_edm`、`D4_care`、`H3_directory_admin`），方便日後討論「哪個功能」時雙方指的是同一件事，不會混淆。

(4) 開發規則——把團隊默契寫下來，讓每一輪對話都遵守

規格書明訂了 7 條硬規則，例如「所有 DB 查詢必須使用 supabase client，不寫 raw SQL」「每個 view 開頭先確認 feature flag」「型別全部從 @/types 匯入，不重複定義」。這些規則的價值在於：即使跨越四十多輪對話、可能由不同 session 接手，程式碼風格跟架構決策仍然保持一致，不會因為「這輪 AI 忘記了」而走鐘。

這份規格書之後怎麼被使用：它不是寫完就丟掉的文件，而是隨著專案演進持續被回頭參照、修正。例如「帳號邀請流程」的設計，在 ARCHITECTURE.md 裡就明確記錄了「2026-07-02 修訂」——原本規格書設計的單向邀請階層（只有地區能邀執秘、只有執秘能邀社長）被推翻，改成更貼近真實需求的三層對等模型。**規格書會過時、會被修正，這是正常的，重點是每次修正都留下記錄跟理由。**

4. 技術架構定案與理由

4.1 角色體系

系統最終定案為三層角色（原規格書的第四層「一般社員」列為 Phase 2 保留）：

角色	說明	生命週期
<code>district_admin</code> 地區管理員	全區檢視，不能編輯各社資料	長期
<code>club_secretary</code> 執行秘書	每社 1 個，社務全權編輯	長期（換人才換帳號）
<code>club_admin</code> 社長	每社 1 個（當屆），社務全權編輯，與執秘對等	一年一屆

為什麼社長跟執秘要對等，而不是階層關係？這是實際踩過坑之後改的（詳見第 6 節案例）。原始設計是「執秘管社長」的單向階層，結果卡在社長沒辦法邀請新執秘、執秘之間也無法互相交接的死結，才改成互為對等的「各社層」。

4.2 貫穿全系統的一條設計原則：各社管各社，地區只管地區

這是整個系統從頭到尾唯一沒有動搖過的原則：每個社只能看到、管理自己社的資料；地區管理員能看到全區彙總，但不能直接改各社的內部資料。

技術實作上，這條原則落實成兩層：

- 每張業務資料表都有 `club_id` 欄位
- Supabase RLS（Row Level Security）在資料庫層自動隔離：`club_id = current_club_id()`，`district_admin` 才能讀全區，其他角色的查詢一律被資料庫強制加上自己的 `club_id` 篩選——就算前端程式碼寫錯、忘記加篩選條件，資料庫也不會放行

這不只是技術設定，更是一種態度：系統應該尊重每個社的自主權，而不是把權力集中到地區。這條原則後來也決定了「跨社活動報名」這類新功能怎麼設計——不是打開一個大水龍頭讓大家互看，而是只在「你自己主辦的活動」這個明確範圍內，才能看到別社社友報名進來的資料。

4.3 Feature Flag：讓地區決定要不要開放某個功能

功能開關分兩層：地區預設（`club_id = NULL`）跟社別覆蓋（特定 `club_id` 蓋過地區預設）。例如 EDM 文案產生器（`B5_edm`）、社友關懷（`D4_care`）這類選配功能，地區管理員可以在後台直接開關，不需要重新部署程式碼。

4.4 資料庫表格清單（Phase 1 完整版）

表格	說明
<code>clubs</code>	各社基本資料
<code>user_profiles</code>	使用者 profile（擴充 <code>auth.users</code> ）
<code>feature_flags</code>	功能開關（地區/社別兩層）
<code>meetings</code>	例會紀錄
<code>attendance_sessions</code> / <code>attendance_details</code>	出席彙總 / 逐人明細
<code>member_attendance_rate</code>	個人出席率（VIEW）
<code>roster</code>	社友名冊
<code>prospective_members</code>	潛在社友
<code>role_permissions</code>	角色權限矩陣（可在 UI 調整）
<code>invite_log</code>	邀請稽核記錄

5. 開發歷程：七個階段的軌跡

四十多輪對話不是均勻分散的流水帳，回頭看可以整理成七個明顯的階段：

階段一 | 地基：帳號、權限、核心四大功能（早期輪次）

從規格書落地成第一版可執行的系統：Email 登入、四級角色、例會管理（B1-B4）、社友名冊（D1-D3）、地區通訊錄（H1-H3）、儀表板。這階段同時做了「細粒度權限系統」——把原本寫死在程式碼裡的角色權限（RLS 用 `role IN (...)`）、前端散落 4 處的 `auth.role === '...'`），改成地區管理員可在 UI 上直接調整、且真正驅動後端 RLS 的權限矩陣（`role_permissions` 表 + `has_permission()` function）。

階段二 | 正式站部署除錯（Cloudflare Pages）

把系統從本機推上正式網址，踩了一輪扎實的部署坑：`tsconfig.node.json` 設定衝突讓 build 本身就失敗、缺 `vite-env.d.ts` 型別檔、Cloudflare Build 設定一開始整個空白（相當於直接把原始碼當靜態檔案丟上去，沒有真的執行 build）、環境變數第一次設定沒生效、SPA 路由 fallback 的新舊語法不相容。這階段最大的教訓是：先前所有 push 上去的版本，很可能從來沒有真正 build 成功過，只是沒人發現。

階段三 | 帳號邀請機制打磨

「邀請帳號」這條路徑最長（前端 → Edge Function → Supabase Auth Admin API → DB trigger → 寄信 → 使用者設密碼 → 重新登入），也是踩坑最密集的一段：Edge Function 的 CORS 處理、`auth.getUser()` 的 token 傳遞方式、`handle_new_user()` 讀錯 metadata 欄位導致的大規模資料錯誤（詳見第 6 節）、Site URL 誤留在 localhost、Gmail SMTP 兩個帳號搞混的地雷。這階段也把權限模型從單向階層改成三層對等（見 4.1）。

階段四 | 降低使用門檻：自助註冊、手機號碼帳號、地區唯讀角色

考量不是每個社友都習慣用 Email、記密碼，加入「手機號碼帳號」機制——帳號密碼都是手機號碼，執秘直接幫社友建好即可用。同時開放自助註冊（依分區選社）、新增「地區（唯讀）」這個第四種存取層級，滿足「有些人需要看到地區資料，但不該有編輯權」的需求。

階段五 | 視覺改版：從左側選單到頂部橫向選單

這階段的特別之處是**不是從零設計**。使用者提供了另一個 repo 裡已經做好的靜態原型（`rotary3481_platform_12.html`，7388 行單檔），裡面已經定義好配色（深藍 `#1C2B4A` / 古銅金 `#B8892A`）跟頂部橫向選單的版面。這輪把這套視覺風格「移植」進實際運作的 Vue 系統：選單邏輯照抄既有的角色/feature-flag 判斷規則，只是排列方式從直向改成橫向可捲動。**善用已經有人想清楚的東西，而不是每次都重新設計，是這個過程很重要的一部分。**

階段六 | 社史資料與跨社協作

新增「友好社」「歷屆社長」「服務計劃總覽」等記錄社史的功能，讓資深社累積下來的文化跟經驗可以被數位化保存、不因為換屆而流失。同時支援「單一帳號協助管理兩個社」的真實情境（例如有社友同時協助兩社行政），一個帳號可以被授權管理不只一個社，切換視角時各社資料仍互不干擾。

階段七 | 社友活動報名系統

最新一輪的功能：社友可以自己上線查詢活動、報名；活動如果是別社主辦，主辦社也能即時看到跨社報名的名單，不用再靠人工彙整。這階段也把「例會」自動同步進活動報名系統，讓例會也能統計「預計出席人數」，但刻意把可見範圍限定在本社（不像一般活動全地區公開），維持跟例會既有隔離規則一致。規劃中的下一步是活動前 7/3/1 天自動發 Email / LINE 提醒。

6. 真實踩過的坑：五個代表性案例

四十多輪對話裡記錄了十幾個踩坑案例，這裡挑五個最有代表性、最能說明「AI 協作開發實際上長什麼樣子」的案例。

案例一：登入後系統「失憶」——RLS 無窮遞迴

現象：地區管理員登入後，選單只剩「地區通訊錄」一項，其他全部消失，即使帳號權限設定明明是對的。畫面不會顯示任何錯誤訊息，只是「看起來壞了但講不出哪裡壞」。

根因：檢查「你是誰」的 SQL function (`current_club_id()` 等) 本身不是 `SECURITY DEFINER`，內部又要查詢 `user_profiles` 表；當這些 function 被用在 `user_profiles` 自己的 RLS 規則裡時，內部查詢又要重新套用同一張表的 RLS，形成無窮遞迴 (Postgres 錯誤碼 `42P17`)。前端因為沒有處理這個錯誤，遞迴失敗被靜默吞掉，角色判斷全部失效。

解法：把這些 helper function 改成 `SECURITY DEFINER` (以 function 擁有者身分執行、略過 RLS)，policy 裡直接寫的子查詢也改成呼叫 function。排查方式後來也被記錄下來：在瀏覽器 console 用 `localStorage` 拿到 access token，直接打 Supabase REST API，繞過前端 store 直接看到資料庫層真正的錯誤訊息——這招後來變成排查類似問題的標準流程。

案例二：`handle_new_user()` 讀錯 metadata 欄位——影響範圍最大的一個 bug

現象：透過邀請流程建立的帳號，`club_id` 全部是 NULL、角色全部 fallback 成一般社員，功能怎麼調都沒用。

根因：Supabase 邀請 API 的 `data` 參數寫入的是 `raw_user_meta_data`，但資料庫觸發器讀的是 `raw_app_meta_data`，兩個欄位對不上。從系統一開始，每一個透過邀請流程建立的帳號都受影響——只是先前一直卡在更前面的問題 (CORS、token 傳遞)，從來沒有真正邀請成功過，所以這個 bug 一直沒被踩到。

解法：修正觸發器改讀正確欄位。但修復不會回頭修正舊資料——已經建立的壞帳號只能刪除重邀。這個案例後來提醒了一件更重要的事：HANDOFF.md 上寫「待辦」的項目，不代表現在還沒完成——後來發現有兩支 Edge Function 早就已經部署完成，只是沒人回來更新記錄。所以「定期用 CLI/資料庫實查核對文件跟實際狀態是否一致」變成一條固定要做的事，不能只憑文件內容當真。

案例三：帳號邀請權限模型設計成單向階層，卡出死結

現象：原本規則是「只有地區能邀執秘」「只有執秘能邀社長」，結果社長沒辦法邀請新執秘，執秘之間也無法互相交接。

根因：這不是程式錯誤，是最初的需求理解跟真實運作方式有落差——規格書寫的時候想像的是階層關係，但實際上社長跟執秘是互相協作的對等角色。

解法：改成三層對等模型（地區 / 各社 / 一般人），社長跟執秘視為同一層，可以互相邀請、互相管理本社帳號。這個案例的重點是：**不是所有 bug 都是程式寫錯，有些是一開始的規格假設就不對，發現之後要願意回頭改架構，而不是在錯的架構上疊補丁。**

案例四：Edge Function 的網址 slug 跟你以為的名字對不上

現象：前端呼叫 Edge Function，回傳看起來像是 CORS 錯誤，但怎麼修 CORS 設定都沒用。

根因：在 Supabase Dashboard 手動建立 function 時，如果當下沒把名字填對，會被塞一個隨機 slug（例如 `rapid-task`）當實際路由。之後在設定頁把「顯示名稱」改掉，並不會改變實際的網址 slug。前端打到不存在的網址，404 沒有 CORS 標頭，瀏覽器把它擋下來，看起來就像 CORS 問題，但其實從頭到尾都打錯地方。

解法：只能刪除重建，建立當下就把名字填對。這個案例說明表面症狀（CORS 錯誤）常常不是真正病因，除錯要一路往下查到底層，不能看到什麼錯誤訊息就往那個方向修。

案例五：Cloudflare Pages 一開始從來沒有真正 build 成功過

現象：程式碼推上 GitHub、Cloudflare Pages 也顯示部署成功，但正式站功能怪怪的、跟本機不一致。

根因：疊了三個問題——① `tsconfig.node.json` 設定衝突讓 `npm run build` 本身就會失敗；② 缺少必要的型別宣告檔；③ Cloudflare Pages 的 Build command / Output directory 設定欄位一開始完全空白，代表 Cloudflare 根本沒有執行建置指令，只是把 repo 裡的原始檔案當靜態網站直接丟上去。三個問題疊在一起，代表**先前所有推上去的版本，很可能從來沒有真正編譯執行過。**

解法：修正 `tsconfig` 設定、補上型別檔、手動在 Cloudflare 後台填入正確的 Build command 跟 Output directory。這個案例的教訓是：**「部署顯示成功」不等於「build 真的有跑」，兩者要分開驗證。**

7. 一路走來的的方法論

回頭看四十多輪對話，幾個反覆出現、值得留下來的做法：

- **先寫規格書，再開始寫程式**：不是打一句話就要 AI 生出成品，而是先花時間想清楚系統邊界、角色、技術選型，寫成 `CODEX_PROMPT.md`，之後每一輪對話都以這份文件為基準討論偏離跟修正。
- **每一輪都留下記錄，不只記錄「做了什麼」，更記錄「為什麼這樣做」跟「捨棄了什麼做法」**：`HANDOFF.md` 不是單純的變更日誌，而是把取捨的理由也寫下來，方便之後任何人（包含換掉的開發者、換屆的幹部）回頭理解系統為什麼長成現在這樣。
- **善用已經想清楚的東西，不從零開始**：視覺改版直接沿用其他人做好的靜態原型；規格書的功能編號系統一路沿用到 feature flag 命名，沒有中途重新發明一套。
- **沒有一次就對，都是先做一版、用真實情境撞出問題，再回頭修**：這跟先畫好全部藍圖再動工的蓋房子邏輯不一樣，更像一邊住一邊調整格局。四十多輪裡幾乎每一輪都是「使用者回報一個真實情境下才會發現的問題」開始的。
- **系統性除錯，不是憑空猜**：遇到「看起來壞了但講不出哪裡壞」的問題，固定會往下一路查：前端 console → Edge Function log → 資料庫直接查詢驗證，找到真正根因才動手改，而不是頭痛醫頭。
- **每個決定都明確寫下「這次沒做什麼」**：例如「沒有加 feature flag」「沒有開放地區視角查詢別社資料」，避免下一輪或其他人誤以為是遺漏，其實是刻意的範圍控制。

8. 現況總覽

目前系統已經在正式站運作（<https://d3481clubmanagementsystem.pages.dev>），涵蓋：

- **帳號與權限**：Email 註冊、手機號碼帳號、邀請制、跨社協作、三級角色（地區管理員 / 執秘 / 社長，另有地區唯讀層級）
- **社務管理**：例會與出席紀錄（含個人出席率統計）、社內公告、幹部名冊、歷屆社長與服務計劃、友好社紀錄
- **地區管理**：全區社團總覽、地區公告、總監獎項統整、通訊錄
- **AI 輔助**：EDM 文案產生器
- **社友活動報名**：跨社活動報名、例會預計出席統計
- 一套功能開關機制，讓地區可以決定要不要開放某個功能給各社使用

全部做好之後，多出來的不是「一套軟體」，而是地區第一次有能力**即時**看到全區的狀況，不用再一個一個社打電話問。

9. 下一步 (Phase 2)

- 活動報名 Email / LINE 到期前自動提醒 (7/3/1 天前，架構已規劃，尚未實作)
- 各社官方 LINE 帳號串接 (人工協助綁定模式)
- 社友關懷 (D4_care)
- 社費 / 財務模組
- 一般人公開瀏覽層 (各社自訂哪些資料對外公開，需要新增公開欄位 + 匿名可讀 RLS + 對外頁面，範圍較大，尚未設計)

10. 給下一個想做類似系統的人

這個系統從第一天就不是為了自己用而已。3481 地區的規模、遇到的問題、走過的坑，很可能其他地區、其他組織也正在經歷。

如果你也在想「我們是不是也能做一套自己的系統」——答案是可以，而且流程比想像中更容易上手：

1. **先寫規格書**，把系統邊界、角色、技術選型想清楚，不用完美，但要明確
2. **一輪一輪對話**，每次解決一個具體問題，不要求一次到位
3. **每一輪都留下記錄**，尤其是「為什麼這樣做」——這比程式碼本身更值得帶走
4. **預期會踩坑**，尤其是部署、帳號驗證這類跨系統整合的環節，坑本身不是失敗，記錄下來就是資產
5. **善用已經做過的東西**，不管是別人做好的視覺原型，還是自己過去某一輪解過的類似問題

你不需要重新經歷「登入後系統失憶」那種 bug，因為別人已經踩過了；你不需要重新想「要不要開放手機號碼登入」，因為別人已經想過、做過、驗證過。